ADDIS ABABA
SCIENCE AND TECHNOLOGY UNIVERSITY
DEPARTMENT OF SOFTWARE ENGINEERING
Database System
Hospital Database Management Sysyem

GROUP 1 ASSIGNMENT | SECTION A

Group members	Id number
Abigiya Tegene	ETS 0065/17
Abigiya Yonas	ETS 0066/17
Dibora Endris	ETS 0441/17
Abubeker Mohammed	ETS0089/17
Amanuel Alemu	ETS 0145/17
Akleshiya Abrham	ETS 0118/17
Abel Birhane	ETS 0034/16

Submitted To:- Dr.Yaynshet M.

Submission Date:- May 22,2026 G.C

Table of Content **Page no**

Introduction

1) Background of the organization

This project is designed around a hypothetical 350-bed specialized hospital located in Addis Ababa, Ethiopia. The hospital is envisioned as a mid-to-large scale healthcare facility. The hospital operates across more than 10 clinical departments including Internal Medicine, Surgery, Pediatrics, Obstetrics & Gynecology, Orthopedics, Neurology and so on. It handles an average of 400-500 outpatient visits and admits approximately 80-120 inpatients at any given time, and performs over 30 surgical procedures daily. The hospital employs roughly 1200 staff members including physicians, nurses, laboratory technicians, pharmacists, and administrative personnel.

The hospital still relies heavily on manual and semi-automated data processing methods for managing patient records, appointment scheduling, laboratory orders, pharmacy inventory, and billing operations. The hospital's current information infrastructure consists of disconnected spreadsheets, paper-based filing systems, and a few stand-alone applications that do not communicate with each other, leading to significant inefficiencies in daily operations.

2) Problem Statement

The hospital faces severe data management challenges that directly impact the quality of healthcare delivery, operational efficiency and patient safety. The hospital currently relies heavily on paper-based records, disconnected spreadsheets, and standalone applications that do not communicate effectively across departments. As a result, critical hospital operations are inefficient, time-consuming, and vulnerable to errors.

Critical Problems Identified

- Fragmented Patient Records: Patient data is scattered across paper files in different departments. When a patient visits multiple departments (e.g., cardiology, laboratory, pharmacy), there is no unified record, leading to duplicate tests, missing history, and potential medication conflicts.
- Inefficient Appointment Scheduling: Appointments are managed manually in logbooks, causing double-booking, long wait times (patients often wait 4–6 hours), and no-show management issues,while doctors lose 20–30% of their scheduled time due to scheduling conflicts.
- Laboratory Result Delays: Laboratory Requests and test results are processed manually using paper forms. Test orders are physically transported to laboratories, and results are delivered manually to doctors or departments. This creates delays of 24-72 hours for routine laboratory tests, slowing diagnosis and treatment processes and negatively affecting patient care.
- Pharmacy Inventory Mismanagement: Medication stock levels are tracked manually, leading to frequent stock-outs of essential drugs, expired medication going undetected, and inability to correlate prescriptions with actual dispensing records.
- Lack of Analytic and Reporting Capabilities: Hospital administrators are unable to generate real-time reports related to bed occupancy, patient statistics, disease prevalence, doctor productivity or financial performance. The absence of centralized data processing makes evidence-based decision-making difficult and limits strategic planning.
- Data Redundancy and Inconsistency: Information of the same patient can be repeatedly entered in multiple departments without proper validation or synchronization. This creates inconsistent and contradictory records, such as different blood types or contact information being recorded for the same patient in different locations.

These challenges collectively result in reduced healthcare quality, increased operational costs, staff frustrations, delays in service delivery, and potential risks to patient safety. Furthermore, the lack of an integrated information system limits the hospital's ability to manage data efficiently and support informed administrative decisions.

To address the identified challenges, the hospital should implement a centralized and integrated Hospital management information system (HMIS) supported by a relational database management system. The proposed system would digitally connect all hospital departments, including registration, laboratory, pharmacy, appointments, billing and administration, through a unified database platform. The integrated system would improve data accessibility, reduce redundancy, enhance operational efficiency, and strengthen data integrity which eventually leads to better healthcare delivery and decision-making processes.

3) Objectives

General objective

To design and implement an integrated database system for the hospital that centralizes patient, medical, administrative, and financial data to improve healthcare delivery, operational efficiency, and decision-making capabilities.

Specific objectives

- To design a relational database schema for managing hospital operations including patients, doctors, appointments, medical records, laboratory tests, prescriptions, and billing.
- To implement the database system in MySQL using proper constraints, relationships, and indexing techniques.
- To design a MongoDB schema for handling semi-structured medical data.
- To create realistic sample datasets and implement SQL and MongoDB queries for hospital reporting and operations.
- To design input forms and output reports based on the database system.
- To test the system for data integrity, performance, and query correctness.
- To compare the relational (MySQL) and document-oriented (MongoDB) approaches for hospital data management.

4) Scope and Benefits

Scope

Scope	In-Scope
Departments	Internal Medicine, Surgery, Pediatrics, Orthopedics, Cardiology, Neurology, Laboratory, Pharmacy
Functions	Patient registration, appointment scheduling, admission/discharge, diagnosis recording, prescription, lab orders, billing
Users	Doctors, nurses, receptionists, lab technicians, pharmacists, billing clerks
Platform	MySQL 8.0+, MongoDB 6.0+
Data	Sample data (5–10 records per entity)

Benefits

Patient Care

- ✓ Unified patient record across all departments
- ✓ Reduced wait times through organized scheduling
- ✓ Faster lab result availability

- ✓ Medication conflict detection

Operational Efficiency
- ✓ Elimination of duplicate data entry
- ✓ Real-time pharmacy stock tracking
- ✓ Automated bed/ward management
- ✓ Streamlined admission/discharge workflow

Decision Support
- ✓ Department workload analytic
- ✓ Disease prevalence reporting
- ✓ Revenue and expense tracking
- ✓ Doctor productivity metrics

Data Integrity
- ✓ Enforced referential integrity
- ✓ Constraint-based data validation
- ✓ Elimination of data redundancy
- ✓ Audit trail capability

5) Methodology and Tools

Development Methodology

This project follows a Waterfall-based Database Development Life cycle (DDLC) approach, adapted from the Standard Systems Development Life Cycle. The methodology consists of five sequential phases:

- Requirements Collection and Analysis: Studying the hospital's operations, interviewing staff (simulated), and documenting data requirements through structured analysis techniques.
- Conceptual Database Design: Creating an Entity-Relationship (ER) model that captures all entities, attributes, relationships and cardinality constraints without regard to implementation specifics.
- Logical Database Design: Translating the ER model into relational schemas and applying normalization (up to BCNF) to eliminate insertion, update, and deletion anomalies.
- Physical Database Design (Implementation): Creating actual database objects (tables/collections) in MySQL and MongoDB, defining data types, constraints, indexes, and inserting sample data.
- Testing and Refinement: Executing queries, verifying data integrity, identifying bugs, documenting results, and proposing improvements.

Tools and Technologies

Category	Tool	Purposes
RDBMS	MySQL 8.0	Relational database implementation
NoSQL	MongoDB 6.0	Document database implementation

ER Modeling	draw.io	Creating ERD
MySQL Client	MySQL Workbench	SQL execution and administration
MongoDB Client	MongoDB Compass	MongoDB GUI and query execution
Version Control	Git / GitHub	Source code management
Communication	Telegram	Group Coordination
Presentation	MS PowerPoint	Final Presentation slides

6)Conceptual Database Design

Entity Identification with their corresponding attributes

Through analysis of the hospital's data processing requirements, the following **16 core entities including associative entities that are** required to resolve many-to-many relationships. were identified, along with their key attributes:

No	Entity	Attributes
1	Department	● dept_id ● dept_name ● Location ● phone
2	Doctor	● doctor_id ● first_name ● last_name ● specialization ● hire_date ● phone ● email
3	Nurse	● nurse_id ● first_name ● last_name ● phone ● email ● shift
4	Patient	● patient_id ● first_name ● last_name ● gender ● dob ● phone ● email ● address ● blood_type ● emergency_contact ● emergency_phone

5	Appointment	● appointment_id ● appt_date ● appt_time ● status ● reason
6	Medical Case	● case_id ● diagnosis ● treatment_plan ● start_date ● end_date ● status
7	Medication	● medication_id ● med_name ● dosage_form ● unit ● description ● stock_qty
8	Medical Record	● record_id ● diagnosis_date ● symptoms ● diagnosis ● notes
9	Room	● room_id ● room_type ● bed_count ● status
10	Ward	● ward_id ● ward_name ● ward_type ● Floor ● capacity
11	Payment	● payment_id ● amount ● payment_date ● payment_method ● status
12	Admission	● admission_id ● admission_date ● discharge_date ● status
13	Prescription	● prescription_id ● frequency ● duration ● quantity
14	Lab Test	● test_id ● test_name ● description ● cost

15	Lab Order	• order_id • order_date • status
16	Lab Result	• result_id • result_value • result_date • is_normal • notes

The relationships of the entities are summarized on the table below.

Relationships Summary

Relationship	**Entities Involved**	**Cardinality**	**Description**
Department employs Doctor	Department↔ Doctor	**1:M**	One department can employ many doctors
Department employs Nurse	Department↔ Nurse	**1:M**	One department can employ many nurses
Ward contains Room	Ward ↔ Room	**1:M**	One ward contains many rooms
Patient treated by doctor	Patient↔ Doctor	**M:N**	A patient can be treated by many doctors and a doctor can treat many patients
Medical_case managed by doctor	Medical_Case↔ Doctor	**M:1**	Many medical cases can be handled by one doctor
Patient books Appointment	Patient↔ Appointment	**1:M**	One patient can book many Appointments
Doctor attends appointment	Doctor↔ Appointment	**1:M**	One doctor can attend many Appointments
Medical_Case belongs to Patient	Medical_Case↔ Patient	**M:1**	Many medical cases can belong to one patient
Patient admitted to room	Patient↔ Room	**M:N**	A patient may stay in many rooms over time and a room may host many patients over time
Doctor supervises Admission	Doctor↔ Admission	**1:M**	One doctor can supervise many admissions
Patient has Medical_Record	Patient ↔ Medical_Record	**1:M**	One patient can have many medical records
Doctor creates Medical_Record	Doctor↔ Medical_Record	**1:M**	One doctor can create many medical records
Admission linked to Medical_Record	Admission↔ Medical_Record	**1:M**	One admission can generate many medical records
Medical_Record contains Medication	Medical_Record↔ Medication	**M:N**	One medical record can include many medications and one medication can appear in many records
Patient requests	Patient↔ Lab_Test	**M:N**	One patient can take many

Lab_Test			lab tests and one lab test can be taken by many patients
Doctor orders Lab_Test	Doctor<-> Lab_Test	**M:N**	One doctor can order many lab tests and one lab test can be ordered by many doctors
Lab_Test produces Lab_Result	Lab_Test<-> Lab_Result	**1:M**	One lab test can produce many results over time
Patient makes Payment	Patient<-> Payment	**1:M**	One patient can make many payments
Admission linked to payment	Admission<-> Payment	**1:M**	One admission can have many payments

After this we build the Entity Relationship Diagram using drawio. Before the physical creation begins the ERD is designed, a visual blueprint used to design, map and organize relational databases. We clearly defined all of our entities, their attributes and relationships. The conceptual design is an important step before going to the logical design.

7)Logical Database Design

After the conceptual database design aka ER diagram implementation, we proceed to logically map the table. To do that we first identified what the attributes are including foreign keys. The ERD should be converted into relational form. This is also called the Logical schema.

Entity	**Arributes**
Department	● dept_id (PK) ● dept_name ● Location ● phone
Doctor	● doctor_id(PK) ● first_name ● last_name ● specialization ● hire_date ● Phone ● Email ● Dept_id(FK)
Nurse	● Nurse_id(PK) ● First_name ● Last_name ● Dept_id(FK) ● Phone ● Email ● shift

Patient	• patient_id(PK) • first_name • last_name • gender • dob • phone • email • address • blood_type • emergency_contact • emergency_phone
Appointment	• appointment_id(PK) • patient_id(FK) • doctor_id(FK) • appt_date • appt_time • status • reason • case_id(FK)
Medical case	• case_id(PK) • patient_id(FK) • doctor_id(FK) • diagnosis • treatment_plan • start_date • end_date • status
Medication	• medication_id(PK) • med_name • dosage_form • unit • description • stock_qty
Medical record	• record_id(PK) • patient_id(FK) • doctor_id(FK) • admission_id(FK) • diagnosis_date • symptoms • diagnosis • notes
Room	• room_id(PK) • room_no • ward_id(FK) • room_type • bed_count • status
Ward	• ward_id(PK) • ward_name • ward_type

	• floor • capacity
Payment	• payment_id(PK) • patient_id(FK) • admission_id(FK) • amount • payment_date • payment_method • status
Admission	• admission_id(PK) • patient_id(FK) • doctor_id(FK) • room_id(FK) • admission_date • discharge_date • status
Prescription	• prescription_id(PK) • record_id(FK) • medication_id(FK) • frequency • duration • quantity
Lab Test	• test_id(PK) • test_name • description • cost
Lab Order	• order_id(PK) • patient_id(FK) • doctor_id(FK) • test_id(FK) • order_date • status
Lab result	• result_id(PK) • order_id(FK) • result_value • result_date • is_normal • notes

8)Normalization (Up to BCNF)

8.1 Introduction to Normalization

After converting the ER diagram into tables the next step will be to implement **normalization,** which is a collection of rules each table should satisfy. Normalization is a series of steps that we follow to obtain **consistent storage** and **efficient access** of

data in relational database. It helps us to reduce **data redundancy** and the **risk of data becoming inconsistent**. The purpose of normalization is to reduce chances for **anomalies** to occur in our database.

8.1.1 Types of Anomalies

Before normalizing it is important to understand the three types of anomalies that can occur in an normalized database:

1. **Insertion Anomaly**
Occurs when we can't add a new piece of data without adding other unrelated data.
Example: Suppose we have a combined table storing patient and department information. If new department "Oncology" is created but has no patients yet, we can't insert the department record because there is no patient id to associate it with.
2. **Deletion Anomaly**
This occurs when a record unintentionally removes other important data.
Example: If we store medication within the prescription table, and a patient's prescription is deleted, we would also lose all information about that medication (name, dosage form, etc.) if no other prescriptions reference it.
3. Modification (Update) Anomaly
Occurs when updating a single piece of data requires updating multiple rows, which is prone to errors.
Hospital Example: If a department's phone number is stored in every Doctor record that belongs to that department, changing the phone number requires updating all doctor records — if we miss even one, the data becomes inconsistent.

8.1.2 Functional Dependency (FD)

Before applying normalization rules, we must understand functional dependency.
Definition: If the existence of something, call it A, implies that B must exist and have a certain value, then we say that "B is functionally dependent on A." We also express this as:
"A determines B"
"B is a function of A"
"A functionally governs B"
"If A, then B"
Notation: A → B (read as "B is functionally dependent on A")
Example: In our hospital database:

- dept_id → dept_name, location, phone (knowing the department ID determines its name, location, and phone)
- doctor_id → first_name, last_name, specialization, dept_id (knowing the doctor ID determines all their attributes)

Important: Functional dependencies are derived from real-world constraints on the attributes, not from sample data.

8.1.3Types of Functional Dependencies

1. Partial Dependency
If an attribute which is **not a member of the primary key** is dependent on some **part of the primary key** (when we have a composite primary key), then that attribute is partially functionally dependent on the primary key.

Notation: If {A, B} is the Primary Key and C is a non-key attribute:

- If {A, B} → C AND (A → C OR B → C), then C is partially dependent on {A, B}

Example: Consider a table with composite key {patient_id, medication_id} storing prescription details:

- If patient_id → patient_name exists, then patient_name is partially dependent on the composite key.

2. Full Dependency
If a non-key attribute is not dependent on any part of the primary key but only on the whole key (when we have a composite primary key), then that attribute is fully functionally dependent on the primary key.
Notation: If {A, B} is the Primary Key and C is a non-key attribute:

- If {A, B} → C AND (A $\nrightarrow$ C AND B $\nrightarrow$ C), then C is fully dependent on {A, B}

Example: In the same prescription table:

- {patient_id, medication_id} → quantity — quantity depends on BOTH patient and medication, not just one.

3. Transitive Dependency
If A functionally determines B, and B functionally determines C, then A transitively determines C — provided that B does not determine A and C does not determine A.
Notation: {(A → B) AND (B → C)} ⇒ A → C, provided that B $\nrightarrow$ A AND C $\nrightarrow$ A
Example :

- doctor_id → dept_id (a doctor belongs to a department)
- dept_id → dept_name, location, phone (department ID determines department details)
- Therefore, doctor_id → dept_name is a transitive dependency

This is exactly why we separate Doctor and Department into different tables.

8.2 First Normal Form (1NF)

Definition:

A table (relation) is in **1NF** if:

1. There are **no duplicated rows** in the table (unique identifier exists)
2. Each cell is **single-valued** (no repeating groups or multi-valued attributes)
3. Entries in a column (attribute) are of the **same kind** (same data type/domain)

How to Achieve 1NF:
There are two ways:

1. Put each repeating group into a **separate table** and connect them with a primary key-foreign key relationship, OR
2. Move repeating groups to new rows by repeating the common attributes

Example: Un normalized Patient Record

PATIENT_ID	FIRST_NAME	PHONE_NUMBERS	DIAGNOSIS	PRESCRIBED_MEDICATIONS
P001	**Abebe**	**0911-123456,0922-654321**	**Hypertension, Diabetes**	**Amlodipine, Metformin**

Problems:

- **phone numbers** contains multiple values (repeating group)
- **diagnoses** contains multiple values
- **prescribed medications** contains multiple values

After 1NF Conversion:

Patient Table:

FIRST_NAME	PHONE_NUMBERS	EMERGENCY_PHONE
Abebe	**0911-123456**	**0922-654321**

MedicalRecord Table:

RECORD_ID	PATIENT_ID	DIAGNOSIS
M001	**P001**	**Hypertension**
M002	**P001**	**Diabetes**

Prescription Table:

PRESCRIPTION_ID	RECORD_ID	MED_NAME
RX001	M001	Amlodipine
RX002	M002	Metfomin

1NF Verification for Our Schema:

All tables in our schema use atomic values. No cell contains multiple values:

- Instead of storing multiple phone numbers in a single field, we provide separate fields (phone, emergency_phone)
- Symptoms and notes are stored as TEXT (a single string), not as arrays or lists
- All repeating groups (multiple prescriptions per record, multiple lab orders per patient) have been extracted into separate tables with foreign key relationships

Schema satisfies 1NF

8.3 Second Normal Form (2NF)

Definition:

A table (relation) is in **2NF** if:

It is in **1NF**, AND

All non-key attributes are **dependent on the entire primary key** (no partial dependencies)

Note: Any table that is in 1NF and has a **single-attribute (non-composite) primary key** is automatically in 2NF.

Example: Table NOT in 2NF

Consider a hypothetical **Appointment_Detail** table with composite key {patient_id, doctor_id}:

PATIENT_ID	DOCTOR_ID	PATIENT_NAME	DOCTOR_NAME	APPT_DATE	REASON
P001	D001	Abebe	Dr.Kebede	2024-01-15	Headache
P001	D002	Abebe	Dr.Ali	2024-01-16	Fever

Functional Dependencies:

- **{patient_id, doctor_id} → apt_date, reason (Full dependency)**
- **Patient_id → patient_name (partial dependency!)**
- **Doctor_id →doctor_name (partial dependency!)**

Problem: If patient "Abebe" changes their name, must update multiple rows.

After 2NF Conversion:

Patient Table:

PATIENT_ID	**PATIENT_NAME**
P001	**Abebe**

Doctor Table:

DOCTOR_ID	**DOCTOR_NAME**
D001	D.Kebede
D002	Dr.Ali

Appointment Table:

PATIENT_ID	**DOCTOR_ID**	**APPT_DATE**	**REASON**
P001	D001	2024-01-15	Headache
P001	D002	2024-01-16	Fever

2NF Verification for Our Schema:

All tables in our schema use **single-column surrogate primary keys**(AUTO_INCREAMNET integers).Therefore:

- There are no composite primary keys
- Consequently, no partial dependencies can exist
- Every non-key attribute functionally dependent on the entire primary key

Schema satisfies 2NF.

8.4 Third Normal Form (3NF)

Definition:

A table (relation) is in 3NF if:

1. It is in 2NF, and
2. There are no transitive dependencies between a primary key and non-primary key attributes (no non-key attribute dependencies on the non-key attribute)

So we have to make all data dependent on nothing but the key.

Example: Table NOT in 3NF

Consider a command Doctor_Dept table:

DOCTOR_ID	DOCTOR_NAME	DEPT_ID	DEPT_NAME	LOCATION
D001	Dr.Kebede	DEPT01	Cardiology	Floor 3
D002	Dr.Ali	DEPT01	Cardiology	Floor 3
D003	Dr.Hana	DEPT02	Surgery	Floor 2

Functional Dependencies:

- doctor_id → doctor_name, dept_id, dept_name, location
- dept_id → dept_name, location (Transitive dependency!)

Transitive Dependency Chain:

- doctor_id → dept_id
- dept_id → dept_name, location
- Therefore: doctor_id → dept_name, location (TRANSITIVE!)

Problem (Modification Anomaly): If Cardiology moves to Floor 4, we must update rows for D001 AND D002. If we miss one, data is inconsistent.

DOCTOR_ID	DOCTOR_NAME	DEPT_ID
D001	Dr.Kebede	DEPT01
D002	Dr.Ali	DEPT01
D003	Dr.Hana	DEPT02

Department Table:

DEPT_ID	DEPT_NAME	LOCATION

DEPT01	Crdiology	Floor 3
DEPT02	Surgery	Floor 2

3NF Verification for Our Schema:

We examined each table for potential dependencies:

TABLE	POTENTIAL TRANSITIVE DEPENDENCY	RESOLUTION
Doctor	Dept_name transitively dependent through dept _id	No, dept_name is stored in the department table, not in Doctor.Doctor only stores dept_id as a foreign key.
Room	Ward_name transitively dependent through ward_id	No,ward_name is in the ward table.
Prescription	Med_name transitively dependent through medication_id	No, med_name is in the Medication table.
LabOrder	Test_name or cost transitively dependent through test_id	No,these are in the LabTest table.
Payment	transitively dependent through	No-amount,payment_date,payment_method, and status all depend directly on payment_id

Schema satisfies 3NF

If a database with all the tables in 3NF, then it is Normalizes Database.

8.5 Boyce-Codd Normal Form (BCNF)

A table(relation) is in **BCNF** if:

1. it is in **3NF**, and
2. For every **non-trival functional dependency** X → Y, X **must be a superkey**(a candidate key or the primary key)

BCNF is a stronger version of 3NF that handles cases where a table has multiple overlapping candidate keys.

Example: Table not in BCNF

DOCTOR_ID	ROOM_ID	DATE	TIME_SLOT
D001	R101	2024-01-15	09:00
D001	R102	2024-01-16	10:00
D002	R101	2024-01-17	09:00

Candidate Keys:

- **{doctor_id,date}**
- **{room_id,date}**

Functional Dependencies:

{doctor_id,date}→ room_id,time_slot

{room_id,date}→doctor_id,time_slot

BUT: doctor_id→ room_id does NOT hold (same doctor can ue different rooms)

This table is in 3NF but not in BCNF because:

- Room_id → doctor_id(for a given date) but room_id aloneis not a superkey

BCNF Verification for Our Schema:

We verified all the functional dependencies across every table:

TABLE	FUNCTIONAL DEPENDENCIES	BCNF CHECK
Departmen t	dept_id→{dept_name.location,phone}	dept_id is PK(superkey)
Doctor	Doctor_id→{dept_name,location,phone }	dept_name has Unique constraint→candidat e key(superkey)
	Doctor_id→{all attributes}	Doctor_id is PK(superkey)
	email → {attributes}	Email has unique constraint →candidate key(superkey)
Room	Room_id → {all attributes}	room_id is PK(superkey)
	Room_numbers → {all attrinutes}	room_number has unique constraint → candidate key(super key)
LabTest	test_id → {all attributes}	Test_id is PK (superkey)
	Test_name→ {all attributes}	test_id has unique constraint → candidate

		key(superkey)
All other tables	Only the surrogate priary key determines all the attributes	No other non-key attributes is a determinant

BCNF satisfied for all tables.

8.6 Summary: Normalization Process Applied

NORMAL FORM	REQUIREMENT	HOW OUR SCHEMA SATISFIES IT
1NF	**No repeating groups;atomic values**	**Multi-valued attributes extracted to separate tables (Prescription, LabResult,etc.)**
2NF	**No partial dependencies**	**All tables use single-column surrogate primary keys**
3NF	**No transitive dependencies**	**Related entity attributes stored in their own tables(Dept in Department, Med in Medimication, etc.)**
BCNF	**Every determinant is a superkey**	**All unique attributes (phone, email, room_nubmer, test_name) are candidate keys**

Result: **The entire schema is in BCNF. No anomalies (insertion, update, or deletion) exist in the normalized schema**.

Relationship after Normalization

The relationship after normalized is done up to 3NF the relationship becomes like the table below. The many to many relationships are simplified to 1:M relationships.

Relationship	Entities Involved	Type	Foreign Key
Department employs Doctor	Department↔ Doctor	**1:M**	Doctor.dept_id
Department employs Nurse	Department↔ Nurse	**1:M**	Nurse.dept_id
Ward contains Room	Ward ↔ Room	**1:M**	Room.ward_id
Patient has Medical_Case	Patient↔ Medical_Case	**1:M**	Medical_Case.patient_id
Doctor handles Medical_Case	Doctor↔ Medical_Case	**1:M**	Medical_Case.doctor_id
Patient books Appointment	Patient↔ Appointment	**1:M**	Appointment.patient_id
Doctor attends	Doctor↔ Appointment	**1:M**	Appointment.doctor_id

appointment			
Medical_Case linked to Appointment	Medical_Case↔ Appointment	**1:M**	Appointment.case_id
Patient admitted through Admission	Patient↔ Admission	**1:M**	Admission.patient_id
Doctor supervises Admission	Doctor↔ Admission	**1:M**	Admission.doctor_id
Room assigned in Admission	Room ↔ Admission	**1:M**	Admission.room_id
Medical_Case linked to Admission	Medical_Case↔ Admission	**1:M**	Admission.case_id
Patient has Medical_Record	Patient↔ Medical_Record	**1:M**	Medical_Record.patient_id
Doctor creates Medical_Record	Doctor↔ Medical_Record	**1:M**	Medical_Record.doctor_id
Admission linked to Medical_Record	Admission↔ Medical_Record	**1:M**	Medical_Record.admission_id
Medical_Record contains prescription	Medical_Record↔ Prescription	**1:M**	Prescription.record_id
Medication used in prescription	Medication↔ Prescription	**1:M**	Prescription.medication_id
Patient requests Lab_Order	Patient↔ Lab_Order	**1:M**	Lab_Order.patient_id
Doctor orders Lab_Order	Doctor↔ Lab_Order	**1:M**	Lab_Order.doctor_id
Lab_Test used in Lab_Order	Lab_Test↔ Lab_Order	**1:M**	Lab_Order.test_id
Lab_Order produces Lab_Result	Lab_Order↔ Lab_Result	**1:1**	Lab_Result.order_id(UNIQUE)
Patient makes Payment	Patient↔ Payment	**1:M**	Payment.patient_id
Admission linked to payment	Admission↔ Payment	**1:M**	Payment.admission_id

9) Database Schema Implementation(MySql Implementation)

The Hospital Management System database was implemented using MySQL. The schema consists of multiple interrelated tables designed to manage hospital operations including patient registration, doctor management, appointments, admissions, medical records, laboratory services, prescriptions, payments, and ward management.

The implementation uses:

- Primary keys for unique identification of records
- Foreign keys to establish relationships between entities
- Unique constraints to prevent duplicate records

- Appropriate data types for efficient storage and data integrity

We created a database called hospital_db and created all the tables.

```
CREATE DATABASE hospital_db;
USE hospital_db;

CREATE TABLE Department(
dept_id INT AUTO_INCREMENT PRIMARY KEY,
dept_name VARCHAR(100) UNIQUE NOT NULL,
location VARCHAR(100),
phone VARCHAR(20)
);
```

The department table stores informations about hospital departments.

- dept_id uniquely identifies each department.
- dept_name stores the department name.
- Location specifies the department location.
- Phone stores contact information

```
CREATE TABLE Doctor(
doctor_id INT AUTO_INCREMENT PRIMARY KEY,
first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50) NOT NULL,
Specialization VARCHAR(100) NOT NULL,
phone VARCHAR(20) UNIQUE,
email VARCHAR(100) UNIQUE,
dept_id INT,
hire_date DATE,
FOREIGN KEY(dept_id) REFERENCES Department(dept_id)
);
```

The doctor table stores doctor's personal and professional details and links each doctor to a department.

- Dept_id creates relationship with the department table.
- Unique constraints prevent duplicate phone numbers and emails.

```
CREATE TABLE Patient(
patient_id INT AUTO_INCREMENT PRIMARY KEY,
first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50) NOT NULL,
gender ENUM('M','F'),
dob DATE,
phone VARCHAR(20) UNIQUE,
email VARCHAR(100) UNIQUE,
address TEXT,
blood_type VARCHAR(3),
emergency_contact VARCHAR(100),
Emergency_phone VARCHAR(20)
);
```

- The patient table contains patient demographic and emergency information.

It stores patient personal and medical identification details, includes emergency contact information and unique constraints ensure unique phone numbers and email addresses.

```
CREATE TABLE Ward(
ward_id INT AUTO_INCREMENT PRIMARY KEY,
ward_name VARCHAR(50) UNIQUE NOT NULL,
ward_type VARCHAR(50),
floor INT,
capacity INT
);
```

- This table stores ward names, types, floor numbers and capacities.

```
CREATE TABLE Room(
room_id INT AUTO_INCREMENT PRIMARY KEY,
room_no VARCHAR(10) UNIQUE,
ward_id INT,
room_type VARCHAR(50),
bed_count INT DEFAULT 1,
status VARCHAR(20),
FOREIGN KEY(ward_id) REFERENCES Ward(ward_id),
UNIQUE(room_no, ward_id)
);
```

- The room tablemanages the hospital room information,

Each room belongs to a ward, bed_count specifies room capacity, and status indicates availability.

```
CREATE TABLE Medical_Case(
case_id INT AUTO_INCREMENT PRIMARY KEY,
patient_id INT NOT NULL,
doctor_id INT NOT NULL,
diagnosis TEXT,
treatment_plan TEXT,
start_date DATE,
end_date DATE,
status VARCHAR(50),
FOREIGN KEY(patient_id) REFERENCES Patient(patient_id),
FOREIGN KEY(doctor_id) REFERENCES Doctor(doctor_id),
UNIQUE(patient_id, doctor_id, start_date)
);
```

- The Medical_Case table records patient diagnosis and treatment information. It represents ongoing or completed medical cases, links patients and doctors, and stores diagnosis.

```
CREATE TABLE Appointment(
appointment_id INT AUTO_INCREMENT PRIMARY KEY,
patient_id INT NOT NULL,
doctor_id INT NOT NULL,
appt_date DATE,
appt_time TIME,
```

```
status VARCHAR(20),
reason TEXT,
case_id INT,
FOREIGN KEY(case_id) REFERENCES Medical_Case(case_id),
FOREIGN KEY(patient_id) REFERENCES Patient(patient_id),
FOREIGN KEY(doctor_id) REFERENCES Doctor(doctor_id),
UNIQUE(patient_id, doctor_id, appt_date, appt_time)
);
```

- The appointment table handles appointment scheduling. It stores appointment dates and times, and prevents duplicate appointments using unique constraints.

```
CREATE TABLE Admission(
admission_id INT AUTO_INCREMENT PRIMARY KEY,
patient_id INT NOT NULL,
doctor_id INT NOT NULL,
room_id INT NOT NULL,
admission_date DATE,
discharge_date DATE,
status VARCHAR(20),
case_id INT NOT NULL,
FOREIGN KEY(case_id) REFERENCES Medical_Case(case_id),
FOREIGN KEY(patient_id) REFERENCES Patient(patient_id),
FOREIGN KEY(doctor_id) REFERENCES Doctor(doctor_id),
FOREIGN KEY(room_id) REFERENCES Room(room_id),
UNIQUE(patient_id, room_id, admission_date)
);
```

- The admission table records patient admissions into rooms, tracks patient room admissions and discharge dates, and links patients, doctors, rooms, and medical cases.

```
CREATE TABLE Medical_Record(
record_id INT AUTO_INCREMENT PRIMARY KEY,
patient_id INT NOT NULL,
doctor_id INT NOT NULL,
admission_id INT,
diagnosis_date DATE,
symptoms TEXT,
diagnosis TEXT,
note TEXT,
FOREIGN KEY(patient_id) REFERENCES Patient(patient_id),
FOREIGN KEY(doctor_id) REFERENCES Doctor(doctor_id),
FOREIGN KEY(admission_id) REFERENCES Admission(admission_id),
UNIQUE (patient_id,doctor_id, diagnosis_date)
);
```

- The Medical_Record table stores detailed diagnosis records: it stores symptoms, diagnosis and doctor notes. It is also connected to admissions and patient records.

```
CREATE TABLE Medication(
medication_id INT AUTO_INCREMENT PRIMARY KEY,
med_name VARCHAR(100) NOT NULL,
```

```
dosage_form VARCHAR(50),
unit VARCHAR(20),
description TEXT,
stock_qty INT DEFAULT 0,
UNIQUE(med_name, dosage_form, unit)
);
```

- The medication table stores medicine information. It maintains edication inventory and dosage information.

```
CREATE TABLE Prescription(
prescription_id INT AUTO_INCREMENT PRIMARY KEY,
record_id INT NOT NULL,
medication_id INT NOT NULL,
frequency VARCHAR(50),
duration VARCHAR(50),
quantity INT,
FOREIGN KEY(record_id) REFERENCES Medical_Record(record_id),
FOREIGN KEY(medication_id) REFERENCES Medication(medication_id),
UNIQUE(record_id, medication_id)
);
```

- The Prescription table stores prescribed medications. It links medications to medical records and stores dosage frequency and duration.

```
CREATE TABLE Lab_Test(
test_id INT AUTO_INCREMENT PRIMARY KEY,
test_name VARCHAR(100) UNIQUE NOT NULL,
description TEXT,
cost DECIMAL(10,2)
);
```

- This table stores laboratory test information. It stores available laboratory tests and associated costs.

```
CREATE TABLE Lab_Order(
order_id INT AUTO_INCREMENT PRIMARY KEY,
patient_id INT NOT NULL,
doctor_id INT NOT NULL,
test_id INT NOT NULL,
order_date DATE,
status VARCHAR(20),
FOREIGN KEY(patient_id) REFERENCES Patient(patient_id),
FOREIGN  KEY(doctor_id) REFERENCES Doctor(doctor_id),
FOREIGN KEY(test_id) REFERENCES Lab_Test(test_id),
UNIQUE(patient_id, doctor_id, test_id, order_date)
);
```

- The Lab_Order table records ordered laboratory tests. It tracks lab test requested for patients.

```
CREATE TABLE Lab_Result(
result_id INT AUTO_INCREMENT PRIMARY KEY,
order_id INT UNIQUE NOT NULL,
```

```
result_value VARCHAR(100),
result_date DATE,
is_normal BOOLEAN,
notes TEXT,
FOREIGN KEY(order_id) REFERENCES Lab_Order(order_id)
);
```

- This table stores laboratory test results: the outcomes and observations.

```
CREATE TABLE payment(
payment_id INT AUTO_INCREMENT PRIMARY KEY,
patient_id INT NOT NULL,
admission_id INT,
amount DECIMAL(10,2),
payment_date DATE,
payment_method VARCHAR(50),
status VARCHAR(20),
FOREIGN KEY(patient_id) REFERENCES Patient(patient_id),
FOREIGN KEY(admission_id) REFERENCES Admission(admission_id),
UNIQUE(patient_id, admission_id, payment_date, amount)
);
```

- The Payment table records hospital payments. It tracks payment transactions related to admissions and services.

```
CREATE TABLE Nurse(
nurse_id INT AUTO_INCREMENT PRIMARY KEY,
first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50) NOT NULL,
phone VARCHAR(20),
email VARCHAR(100),
dept_id INT,
shift VARCHAR(20),
FOREIGN KEY(dept_id) REFERENCES Department(dept_id),
UNIQUE(phone),
UNIQUE(email)
);
```

- The nurse table stores nurse information: their detailed information and departmental assignments. It prevents duplicate phone numbers and email addresses.

In conclusion, The implemented database schema provides a comprehensive relational structure for managing hospital operations. Relationships between tables ensure data consistency and integrity through foreign key constraints. The schema supports efficient handling of patient care, appointments, admissions, prescriptions, laboratory services, staffing, and financial transactions within the hospital management system.

10)MongoDB Database Design

To enhance the relational database system, the hospital management platform also utilizes a MongoDB-based NoSQL architecture for managing semi-structured and

hierarchical healthcare data. MongoDB was chosen for its capacity to accommodate nested documents, embedded arrays, and dynamic medical data that can differ from patient to patient. In contrast to relational databases that impose strict table structures, MongoDB permits related information to be consolidated within a single document, which is ideal for storing patient histories, prescriptions, billing details, and lab results.

The MongoDB design for this project features three primary collections: patients, staff, and inventory. These collections were created to reflect essential hospital functions while showcasing the benefits of document-oriented database systems. The patients collection maintains detailed information about individuals, which includes their personal data, emergency contacts, medical history, admissions, lab results, prescriptions, and billing details, all organized within embedded documents and arrays. This setup enables efficient retrieval of all patient-related information through a single query, minimizing the necessity for complex joins that are typical in relational databases.

The staff collection holds data pertinent to hospital personnel such as doctors and nurses. The records encompass identifiers for staff members, their professional roles, departmental assignments, areas of specialization, work schedules, and contact details. Given that staff information can vary based on roles, MongoDB's flexible schema allows for efficient management of differing attributes without the need to restructure the database.

The inventory collection oversees the information related to pharmacy and medication stock. It includes details about medications, such as their names and dosage forms, quantity in stock, and unit cost. This collection supports inventory tracking and automatic stock updates whenever medications are dispensed to patients.

✓ Collection Structure Examples:

```
{
 "patient_id": 101,
 "first_name": "Abebe",
 "last_name": "Bikila",
 "gender": "M",
 "blood_type": "A+",

 "emergency_contact": {
  "name": "Meseret Bekele",
  "phone": "+251911223344"
 },

 "medical_history": [
  {
   "diagnosis": "Hypertension",
   "treatment": "Medication",
   "date": "2026-01-15"
  }
 ],

 "prescriptions": [
```

```
    {
      "medication": "Amoxicillin",
      "quantity": 10,
      "duration": "5 days"
    }
  ]
}
```

✓ Staff Collection Examples:

```
{
  "staff_id": 201,
  "name": "Dr. Hana Tesfaye",
  "role": "Doctor",
  "department": "Cardiology",
  "specialization": "Heart Disease",
  "phone": "+251900112233"
}
```

✓ Inventory Collection Examples:

```
{
  "medication_id": 301,
  "med_name": "Paracetamol",
  "dosage_form": "Tablet",
  "stock_qty": 500,
  "unit_cost": 5.50
}
```

CRUD Operations Example:

```
db.patients.insertOne({
  patient_id: 101,
  first_name: "Abebe",
  last_name: "Bikila"
})
```

Inserts one patient.

```
db.patients.find({ blood_type: "A+" })
```

It helps us find patients with blood type A+.

```
db.inventory.updateOne(
  { med_name: "Paracetamol" },
  { $set: { stock_qty: 450 } }
)
```

This updates one medication’s information.

Advantages of using MongoDB

The MongoDB implementation provides several advantages within the hospital management system:

Flexible Schema Design : MongoDB supports dynamic document structures, making it easier to handle varying medical records and patient histories.

Efficient Storage of Hierarchical Data: Nested documents allow related patient information such as prescriptions, lab results, and admissions to be stored together.

Faster Data Retrieval : Embedded documents reduce the need for complex joins, improving query performance for patient-centered operations.

Scalability : MongoDB supports horizontal scaling and can efficiently manage growing hospital datasets and increasing patient records.

Real-Time Data Updates : Operations such as inventory stock reduction and billing updates can be processed quickly in real time.

11) Input Forms, Outputs, and Reports

11.1. Input Forms

Form 1: Patient Registration

First Name *
e.g., Abebe
Last Name *
e.g., Kebede
Gender *
Male / Female
Date of Birth *
YYYY-MM-DD
Phone
+251-XXX-XXXXXX
Email
abebe@email.com
Address
Bole, Addis Ababa
Blood Type
A+ / A- / B+ / B- / AB+ / AB- / O+ / O-
Emergency Contact
Name of contact person
Emergency Phone
+251-XXX-XXXXXX
* Required fields — Maps to: INSERT INTO Patient (first_name, last_name, gender, dob ...)

Form 2: Appointment Booking

Patient ID *
Auto-fill or search by name
Doctor *
Dropdown: Dr. [Name] — [Specialization]
Appointment Date *
YYYY-MM-DD

Appointment Time *
HH:MM (e.g., 09:00)
Reason for Visit
e.g., Persistent headache for 3 days
Maps to: INSERT INTO Appointment (patient_id, doctor_id, appt_date, appt_time, status, reason)

Form 3: Patient Admission

Patient ID *
Auto-fill from registration
Assigned Room *
Dropdown: Available rooms only
Attending Doctor *
Dropdown: Doctors in relevant dept
Admission Date *
YYYY-MM-DD (default: today)
Maps to: INSERT INTO Admission + UPDATE Room SET status='Occupied'

Form 4: Prescription Entry

Medical Record ID *
Link to existing diagnosis record
Medication *
Dropdown: From medication inventory
Frequency *
e.g., 3 times daily, once daily, Every 8 hours
Duration *
e.g., 7 days, 2 weeks, 1 month
Quantity *
e.g., 21 (tablets)
Maps to: INSERT INTO Prescription + UPDATE Medication SET stock_qty = stock_qty - quantity

11.2.Output Reports

Report 1: Patient Medical Summary

QUERY-BASED
Consolidated view of a patient's personal information, all appointments, admission history, diagnoses, prescriptions, and lab results.

Patient profile	appointment	Admissions	diagnosis	prescriptions	Lab result
Abebe Kebede M, 35yr, O+	3 total 2 completed	1 active Room 204	Hypertension Type 2 Diabetes	Amlodipine 5mg Metformin 500mg	Fasting Glucose: 145 mg/dL Abnormal

Report 2: Department Workload Analysis

AGGREGATION

Summary statistics per department: number of doctors, appointments handled, active admissions, and total revenue.

department	Doctors	Appointement	Active admissions	Total Revenue(ETB)
Internal Medicine	5	142	18	285,000
Surgery	4	87	12	520,000
Pediatrics	3	98	15	178,500
Cardiology	3	64	8	312,000
Orthopedics	2	53	6	245,000

Report 3: Bed Occupancy Report

STATUS QUERY

Ward	Total rooms	Total beds	occupied	Available	Occupancy rate
General Ward A	10	20	16	4	80%
General Ward B	8	16	10	6	63%
General Ward C	6	6	5	1	83%
General Ward D	4	4	4	0	100%

Conclusion

In conclusion, the Hospital Database Management System developed in this project provides an efficient and integrated solution for managing hospital operations. The MySQL relational database ensures data data consistency, integrity, and reliable management of patients, staff, appointments, admissions, laboratory services, prescriptions, and billing records.

The MongoDB implementation supports flexible storage of semi-structured healthcare data such as patient histories, prescriptions, and lab results using embedded documents and dynamic schema.

Overall, the project demonstrates how combining relational and NoSQL databases can improve healthcare management, operational efficiency, and decision-making while providing a strong foundation for future system enhancements.

References and Links

[1] Oracle Corporation, *MySQL 8.0 Reference Manual*. [Online]. Available: MySQL Documentation. [Accessed: May 22, 2026].

[2] MongoDB Inc., *MongoDB Documentation*. [Online]. Available: MongoDB Documentation. [Accessed: May 22, 2026].

[3] diagrams.net, *draw.io Diagramming Tool*. [Online]. Available: draw.io Official Website. [Accessed: May 22, 2026].

[4] Fundamentals of Database Systems, 7th ed. Pearson Education, 2016.

[5] Database System Concepts, 7th ed. McGraw-Hill Education, 2019.

[6] GitHub Inc., *GitHub Documentation*. [Online]. Available: GitHub Official Website. [Accessed: May 22, 2026].

[7] MongoDB Inc., *MongoDB Compass Documentation*. [Online]. Available: MongoDB Compass Documentation. [Accessed: May 22, 2026].

[8] Oracle Corporation, *MySQL Workbench Documentation*. [Online]. Available: MySQL Workbench Documentation. [Accessed: May 22, 2026].